Sri Lanka Institute of Information Technology

Faculty of Computing

SE3062 – Intelligent Systems

Year 3 · 2026

Search Algorithms in Pac-Man

Group Report

Group ID: IA_KDY_AI_01

Group Members

IT24104191 – Raashidh M.R

IT24103933 – Atheek M.F

IT24103352 – Ahamed M.A.W

IT24100602 – Wijesiri S.P.R.H

Lecturer: Mr. Jeewaka Perera

Public Git Repository

https://github.com/Raashidh-Rizvi/Pac-Man-Intelligent-Search

1. Introduction

This project implements the search component of the Berkeley Pac-Man framework. Only `search.py` (Q1–Q4) and `searchAgents.py` (Q5–Q7) were edited; `util.py`, `pacman.py`, `game.py`, `autograder.py` and `test_cases/` are unchanged, and no file, function or class was renamed. Data structures come from `util.py`: Stack (DFS), Queue (BFS) and PriorityQueue (UCS, A*). The output blocks below are the real terminal output of `python autograder.py -q qN --no-graphics` for each question; the score line uses the local autograder's weights (see Section 3).

2.1 Q1 – Depth First Search

Author: Wijesiri S.P.R.H (IT24100602)

Code edited/added: `search.py: depthFirstSearch(problem)`

DFS follows one path as deep as possible before backtracking, so the fringe is a LIFO `util.Stack`. Each entry holds a state and the list of actions that reached it. The start state is pushed with an empty path. Each iteration pops an entry and tests it with `problem.isGoalState`, returning the actions for a goal. Otherwise, if the state is not in the `visited` set, it is added and every successor from `getSuccessors` is pushed with its action appended. The visited set makes this graph search, so cycles cannot loop forever and no state is expanded twice. DFS is complete on finite graphs but not optimal: on mediumMaze it returns a 130-step path although the shortest has 68 steps (146 nodes expanded).

Autograder output (text)

```
Question q1
===========
*** PASS: test_cases\q1\graph_backtrack.test
***   solution:    ['1:A->C', '0:C->G']
***   expanded_states:  ['A', 'D', 'C']
*** PASS: test_cases\q1\graph_bfs_vs_dfs.test
***   solution:    ['2:A->D', '0:D->G']
***   expanded_states:  ['A', 'D']
*** PASS: test_cases\q1\graph_infinite.test
***   solution:    ['0:A->B', '1:B->C', '1:C->G']
***   expanded_states:  ['A', 'B', 'C']
*** PASS: test_cases\q1\graph_manypaths.test
***   solution:    ['2:A->B2', '0:B2->C', '0:C->D', '2:D->E2', '0:E2->F', '0:F->G']
***   expanded_states:  ['A', 'B2', 'C', 'D', 'E2', 'F']
*** PASS: test_cases\q1\pacman_1.test
***   pacman layout:    mediumMaze
***   solution length: 130
***   nodes expanded:    146
### Question q1: 3/3 ###
```

Autograder screenshot

[INSERT SCREENSHOT HERE]

Screenshot of: python autograder.py -q q1 --no-graphics (showing Question q1 score)

2.2 Q2 – Breadth First Search

Author: Atheek M.F (IT24103933)

Code edited/added: `search.py: breadthFirstSearch(problem)`

BFS expands the shallowest nodes first, so the fringe is a FIFO `util.Queue`. The start state is enqueued with an empty action list. Each iteration dequeues the oldest entry, applies the goal test and, if the state has not been expanded, records it in a `visited` set and enqueues all successors with extended action lists. Because every Pac-Man move costs 1, the first goal dequeued is reached by a path with the fewest actions, so BFS is complete and optimal here. On mediumMaze it finds the 68-step optimum (269 nodes expanded). The cost is memory: the queue holds a whole layer of the search tree.

Autograder output (text)

```
Question q2
===========
*** PASS: test_cases\q2\graph_backtrack.test
***   solution:    ['1:A->C', '0:C->G']
***   expanded_states:  ['A', 'B', 'C', 'D']
*** PASS: test_cases\q2\graph_bfs_vs_dfs.test
***   solution:    ['1:A->G']
***   expanded_states:  ['A', 'B']
*** PASS: test_cases\q2\graph_infinite.test
***   solution:    ['0:A->B', '1:B->C', '1:C->G']
***   expanded_states:  ['A', 'B', 'C']
*** PASS: test_cases\q2\graph_manypaths.test
***   solution:    ['1:A->C', '0:C->D', '1:D->F', '0:F->G']
***   expanded_states:  ['A', 'B1', 'C', 'B2', 'D', 'E1', 'F', 'E2']
*** PASS: test_cases\q2\pacman_1.test
***   pacman layout:    mediumMaze
***   solution length: 68
***   nodes expanded:    269
### Question q2: 3/3 ###
```

Autograder screenshot

[INSERT SCREENSHOT HERE]

Screenshot of: python autograder.py -q q2 --no-graphics (showing Question q2 score)

2.3 Q3 – Uniform Cost Search

Author: Atheek M.F (IT24103933)

Code edited/added: `search.py: uniformCostSearch(problem)`

UCS generalises BFS to varying step costs by always expanding the fringe node with the lowest path cost g(n). The fringe is a `util.PriorityQueue` whose priority is the cumulative cost; each entry stores the state, the action list and the cost. The goal test is done when a node is *dequeued*, not when generated, because a cheaper path to the goal may still be on the fringe. For each successor the new cost is `cost + stepCost`. A visited set prevents re-expansion. With non-negative step costs UCS is optimal. It solves mediumMaze with cost 68, and with the cost functions in `searchAgents.py` it finds cheap paths for StayWestSearchAgent and expensive-but-forced paths for StayEastSearchAgent.

Autograder output (text)

```
Question q3
===========
*** PASS: test_cases\q3\graph_backtrack.test
***   solution:    ['1:A->C', '0:C->G']
***   expanded_states:  ['A', 'B', 'C', 'D']
*** PASS: test_cases\q3\graph_bfs_vs_dfs.test
***   solution:    ['1:A->G']
***   expanded_states:  ['A', 'B']
*** PASS: test_cases\q3\graph_infinite.test
***   solution:    ['0:A->B', '1:B->C', '1:C->G']
***   expanded_states:  ['A', 'B', 'C']
*** PASS: test_cases\q3\graph_manypaths.test
***   solution:    ['1:A->C', '0:C->D', '1:D->F', '0:F->G']
***   expanded_states:  ['A', 'B1', 'C', 'B2', 'D', 'E1', 'F', 'E2']
*** PASS: test_cases\q3\ucs_0_graph.test
***   solution:    ['Right', 'Down', 'Down']
***   expanded_states:  ['A', 'B', 'D', 'C', 'G']
*** PASS: test_cases\q3\ucs_1_problemC.test
***   pacman layout:    mediumMaze
***   solution length: 68
***   nodes expanded:    269
*** PASS: test_cases\q3\ucs_2_problemE.test
***   pacman layout:    mediumMaze
***   solution length: 74
***   nodes expanded:    260
*** PASS: test_cases\q3\ucs_3_problemW.test
***   pacman layout:    mediumMaze
***   solution length: 152
***   nodes expanded:    173
*** PASS: test_cases\q3\ucs_4_testSearch.test
***   pacman layout:    testSearch
***   solution length: 7
***   nodes expanded:    14
*** PASS: test_cases\q3\ucs_5_goalAtDequeue.test
```

```
***   solution:    ['1:A->B', '0:B->C', '0:C->G']
***   expanded_states:  ['A', 'B', 'C']
### Question q3: 3/3 ###
```

Autograder screenshot

[INSERT SCREENSHOT HERE]

Screenshot of: python autograder.py -q q3 --no-graphics (showing Question q3 score)

2.4 Q4 – A* Search

Author: Ahamed M.A.W (IT24103352)

Code edited/added: `search.py: aStarSearch(problem, heuristic)`

A* orders the fringe by f(n) = g(n) + h(n), where g is the path cost so far and h is the heuristic estimate to the goal. A `util.PriorityQueue` is keyed on f, and a dictionary keeps the best g found for each state. A successor is pushed (or its priority lowered with `frontier.update`) only if it improves the best known cost. The goal test occurs on dequeue and a visited set avoids re-expansion. With an admissible and consistent heuristic such as Manhattan distance, A* returns an optimal path. On mediumMaze it finds the 68-step optimum while expanding 221 nodes, fewer than UCS (269). When no heuristic is given, `nullHeuristic` makes it behave like UCS.

Autograder output (text)

```
Question q4
===========
*** PASS: test_cases\q4\astar_0.test
***   solution:    ['Right', 'Down', 'Down']
***   expanded_states:  ['A', 'B', 'D', 'C', 'G']
*** PASS: test_cases\q4\astar_1_graph_heuristic.test
***   solution:    ['0', '0', '2']
***   expanded_states:  ['S', 'A', 'D', 'C']
*** PASS: test_cases\q4\astar_2_manhattan.test
***   pacman layout:    mediumMaze
***   solution length: 68
***   nodes expanded:    221
*** PASS: test_cases\q4\astar_3_goalAtDequeue.test
***   solution:    ['1:A->B', '0:B->C', '0:C->G']
***   expanded_states:  ['A', 'B', 'C']
*** PASS: test_cases\q4\graph_backtrack.test
***   solution:    ['1:A->C', '0:C->G']
***   expanded_states:  ['A', 'B', 'C', 'D']
*** PASS: test_cases\q4\graph_manypaths.test
***   solution:    ['1:A->C', '0:C->D', '1:D->F', '0:F->G']
***   expanded_states:  ['A', 'B1', 'C', 'B2', 'D', 'E1', 'F', 'E2']
### Question q4: 3/3 ###
```

Autograder screenshot

[INSERT SCREENSHOT HERE]

Screenshot of: python autograder.py -q q4 --no-graphics (showing Question q4 score)

2.5 Q5 – Corners Problem (state representation)

Author: Wijesiri S.P.R.H and Ahamed M.A.W

Code edited/added: `searchAgents.py: CornersProblem.getStartState, isGoalState, getSuccessors`

The state must remember where Pac-Man is and which corners are done, so we use `(position, visitedCorners)`, where `visitedCorners` is a tuple of four booleans, one per corner. The tuple is immutable and hashable so it can be stored in the visited set. `getStartState` marks a corner True if Pac-Man starts on it. `isGoalState` returns `all(state[1])`. `getSuccessors` tries the four directions, skips walls, and when the new position is a corner sets its flag to True; every move costs 1. Only the position and four flags are stored (not the food grid or the path), keeping the state space small. BFS on tinyCorners finds the 28-step optimum.

Autograder output (text)

```
Question q5
===========
*** PASS: test_cases\q5\corner_tiny_corner.test
***    pacman layout:    tinyCorner
***    solution length:    28
### Question q5: 3/3 ###
```

Autograder screenshot

[INSERT SCREENSHOT HERE]

Screenshot of: python autograder.py -q q5 --no-graphics (showing Question q5 score)

2.6 Q6 – Corners Heuristic

Author: Raashidh M.R (IT24104191)

Code edited/added: `searchAgents.py: cornerMazeDistances(problem), cornersHeuristic(state, problem)`

The heuristic is the length of the shortest tour that starts at Pac-Man and visits every unvisited corner, using true *maze* distances. A helper runs BFS from each corner once and caches the tables on the problem object. `cornersHeuristic` tries every permutation of the outstanding corners (at most 4! = 24), sums the maze distance from Pac-Man to the first

corner and between consecutive corners, and returns the minimum. **Admissible:** any real plan visits all outstanding corners in some order and each leg costs at least the maze distance, so no plan beats the best ordering. **Consistent:** one move changes the position by one step, so the best tour cost changes by at most 1, giving h(n) <= 1 + h(n'). mediumCorners: 106 steps, 189 nodes expanded (full marks need fewer than 1200).

Autograder output (text)

```
Question q6
===========
*** PASS: heuristic value less than true cost at start state
*** PASS: heuristic value less than true cost at start state
*** PASS: heuristic value less than true cost at start state
path: ['North', 'East', 'East', 'East', 'East', 'North', 'North', 'West', 'West', 'West', 'West', 'North', 'North', 'North', 'N
path length: 106
*** PASS: Heuristic resulted in expansion of 189 nodes
### Question q6: 3/3 ###
```

Autograder screenshot

[INSERT SCREENSHOT HERE]

Screenshot of: python autograder.py -q q6 --no-graphics (showing Question q6 score)

2.7 Q7 – Food Heuristic

Author: Raashidh M.R (IT24104191)

Code edited/added: `searchAgents.py: foodHeuristic(state, problem)`

The state is `(position, foodGrid)`. The heuristic relaxes the problem: it is the maze distance from Pac-Man to the *nearest* remaining food plus the weight of a **minimum spanning tree** (Prim) over all remaining food, with maze distances as edge weights. Any plan must reach at least one food and then connect all the food, and a spanning tree is the cheapest connected structure over those points, so the value never exceeds the true cost (admissible). It is consistent because moving or eating changes it by at most one step. Maze distances come from a BFS helper whose results are cached in `problem.heuristicInfo`. If no food remains the heuristic is 0. trickySearch: 60 steps, 255 nodes expanded (full marks need fewer than 9000).

Autograder output (text)

```
Question q7
===========
*** PASS: test_cases\q7\food_heuristic_1.test
*** PASS: test_cases\q7\food_heuristic_10.test
*** PASS: test_cases\q7\food_heuristic_11.test
*** PASS: test_cases\q7\food_heuristic_12.test
*** PASS: test_cases\q7\food_heuristic_13.test
*** PASS: test_cases\q7\food_heuristic_14.test
*** PASS: test_cases\q7\food_heuristic_15.test
*** PASS: test_cases\q7\food_heuristic_16.test
*** PASS: test_cases\q7\food_heuristic_17.test
*** PASS: test_cases\q7\food_heuristic_2.test
*** PASS: test_cases\q7\food_heuristic_3.test
*** PASS: test_cases\q7\food_heuristic_4.test
*** PASS: test_cases\q7\food_heuristic_5.test
*** PASS: test_cases\q7\food_heuristic_6.test
*** PASS: test_cases\q7\food_heuristic_7.test
*** PASS: test_cases\q7\food_heuristic_8.test
*** PASS: test_cases\q7\food_heuristic_9.test
*** PASS: test_cases\q7\food_heuristic_grade_tricky.test
***   expanded nodes: 255
***   thresholds: [15000, 12000, 9000, 7000]
### Question q7: 5/4 ###
```

Autograder screenshot

[INSERT SCREENSHOT HERE]

Screenshot of: python autograder.py -q q7 --no-graphics (showing Question q7 score)

3. Experimental Summary

Q	Command (layout)	Result	Nodes expanded	Local score
Q1	dfs, mediumMaze	cost 130	146	3/3
Q2	bfs, mediumMaze	cost 68	269	3/3
Q3	ucs, mediumMaze	cost 68	269	3/3
Q4	astar + manhattan, mediumMaze	cost 68	221	3/3
Q5	bfs + CornersProblem, tinyCorners	cost 28	-	3/3
Q6	AStarCornersAgent, mediumCorners	cost 106	189 (limit 1200)	3/3
Q7	AStarFoodSearchAgent, trickySearch	cost 60	255 (limit 9000)	5/4

The local autograder prints Berkeley's original weights; Q7 can show 5/4 because it adds one extra point for every node-count threshold beaten (15000, 12000, 9000, 7000). The assignment weights are 4 marks each for Q1–Q4 and 8 marks each for Q5–Q7 (40 in total). Q6 and Q7 are graded on nodes expanded and both results are far below the full-marks limits. Q8 (ClosestDotSearchAgent) is part of the provided skeleton but is not assessed.

4. Public Git Repository

https://github.com/Raashidh-Rizvi/Pac-Man-Intelligent-Search

Branches: `main`, `feature/q1-dfs-member1`, `feature/q2-q3-atheek`, `feature/q4-q5-member3`, `feat(heuristic)--implement-admissible-corners-heuristic-for-Q6`.

5. Git Contribution Evidence

Commit counts per author (from `git shortlog`):

Author	Commits
Raashidh M.R	18
Atheek M.F	8
Wijesiri S.P.R.H	5
Ahamed M.A.W	4

Recent commit history (`git log --all`):

```
118f3b6 2026-10-11 raashidh rizvi: refactor: add type hints to search algorithms, update search
33e306c 2026-10-09 Atheek_Fareez: Merge pull request #1 from Raashidh-Rizvi/feature/q2-q3-athee
6d0a263 2026-10-09 Atheek-Fareez: docs(report): update Q3 screenshots in REPORT_OUTLINE.md
e77defc 2026-10-09 Atheek-Fareez: feat(search): implement UCS algorithm with path cost tracking
1e8118a 2026-10-09 Atheek-Fareez: merge: resolve merge conflicts with origin/main and preserve
e00cdfb 2026-10-09 raashidh rizvi: docs: assign Raashidh Rizvi (IT24104191) to Q6 and Q7 in tea
eaf21e2 2026-10-08 raashidh rizvi: docs: add ASSIGNMENT_SPECIFICATION.html for PDF printing and
387f93f 2026-10-08 raashidh rizvi: docs: update README with SE3062 details, autograder status,
8747149 2026-10-08 raashidh rizvi: feat: merge feature/q4-q5-member3 (Wazni Ahamed) into main
e92fa45 2026-10-08 raashidh rizvi: feat: merge feature/q1-dfs-member1, feature/q2-q3-atheek, an
207969b 2026-10-08 raashidh rizvi: feat: initialize Pac-Man intelligent search project with gam
```

```
f145ab0 2026-10-07 Raashidh Rizvi: Replaces the Q6 `cornersHeuristic` with a minimum-over-order
50174dc 2026-10-07 WAZNI AHAMED (IT24103352): test(q4-q5): verify Member 3 work on Python 3.11
b029a05 2026-10-07 WAZNI AHAMED (IT24103352): docs(team): correct Member 3 Q4-Q5 ownership
9948757 2026-10-07 WAZNI AHAMED (IT24103352): docs(q5): document corner-state contract and veri
81faccf 2026-10-07 WAZNI AHAMED (IT24103352): fix(search-agent): forward selected heuristic to
9921fc9 2026-10-06 Wijesiri S.P.R.H: docs(q5): add Corners Problem report and test evidence
76ae172 2026-10-06 Wijesiri S.P.R.H: docs(report): update Member 1 contribution
d3cd78d 2026-10-06 Wijesiri S.P.R.H: docs(report): add q1 evidence
43e6cde 2026-10-06 Wijesiri S.P.R.H: Revert "Implement DFS graph search for Q1"
812c731 2026-10-06 Wijesiri S.P.R.H: Implement DFS graph search for Q1
8beeb13 2026-10-06 Atheek-Fareez: feat(search): implement BFS with step-by-step comments and ad
bd97995 2026-10-06 Atheek-Fareez: feat(search): implement BFS algorithm using FIFO Queue
5830d76 2026-10-06 Atheek-Fareez: docs: add student ID IT24103933 for Atheek Fareez
88a305e 2026-10-06 Atheek-Fareez: docs: assign Atheek Fareez to Member 2 (Q2 and Q3)
```

Commit history screenshot

[INSERT SCREENSHOT HERE]

GitHub commit history (Commits page, all branches)

Contribution graph screenshot

[INSERT SCREENSHOT HERE]

GitHub Insights - Contributors / contribution graph showing all four members

6. Individual Contribution Table

Name	Student ID	Questions / tasks	Branch
Wijesiri S.P.R.H	IT24100602	Q1 (DFS); Q5 co-author (Corners Problem); report evidence	feature/q1-dfs-member1
Atheek M.F	IT24103933	Q2 (BFS), Q3 (UCS)	feature/q2-q3-atheek
Ahamed M.A.W	IT24103352	Q4 (A*); Q5 co-author; heuristic forwarding fix; verification	feature/q4-q5-member3
Raashidh M.R	IT24104191	Q6 (corners heuristic), Q7 (food heuristic); repository setup, merging, documentation	Q6 heuristic branch / main

7. AI Usage Declaration

AI tools used: ChatGPT (OpenAI) and Claude Code (Anthropic Claude).

7.1 ChatGPT – planning and work division. The group used ChatGPT to get ideas on how to approach the assignment: to understand the overall assignment and the Pac-Man search tasks, and to decide how to divide the questions between the group members. We analysed its suggestions together and then divided the work between the four members as shown in Section 6 (Q1 – Wijesiri; Q2 and Q3 – Atheek; Q4 – Ahamed; Q5 – Ahamed and Wijesiri; Q6 and Q7 – Raashidh). ChatGPT was used only for ideas and planning, not to write the submitted code.

Prompts used with ChatGPT:

[Paste the exact ChatGPT prompts here.]

7.2 Claude Code – verification and documentation. Claude Code was used to check the project against the assignment brief, verify that only `search.py` and `searchAgents.py` differ from the provided skeleton, restore the skeleton's original names, signatures and docstrings, update the documentation, and generate this report. All results quoted here come from running the autograder and `pacman.py` on our code. The prompts given to Claude Code were:

```
Prompts given to Claude Code (user messages, lightly trimmed for length):
1. "I want you to compare these files ... list all the changes that is done in these things"
2. "make sure only the search.py and searchAgents.py is edited, others aren't edited"
3. "This is the full assignment ... check if any issues are there let me know"
4. "there shouldn't be any new files other than the Python files"
5. "If the function names are changed make sure to rename the functions"
6. "yes restore the type hints, imports and docstrings too"
7. "yes restore search.py the same way"
8. "When running python pacman.py what is the expected behavior of this assignment"
9. "make sure you update the read me files and docs as well according to the assignment"
10. "why is question 6 getting one mark greater than expected ... is it getting 26 out of 25"
11. "Where did the question 8 come from"
12. "is everyone's work done for member 1, 2, 3 and 4"
13. "why Q6 expands 189 nodes and Q7 expands 255 nodes"
14. "I want you generate me a full PDF so that I can submit a report"
```